Lab 00 — Questions

Answer without going back to the theory. One to five sentences is plenty — the reasoning matters, not the vocabulary.

Question 1 [Comprehension]

A binary compiled for Linux (say, `nginx`) runs identically on Ubuntu, Debian and Alpine, yet not at all on Windows without WSL. What does the binary actually need from its system? Why does the distribution make no difference while the kernel makes all of it?

Question 2 [Analysis]

You run `sleep 300 &` in a terminal, then close that terminal. A little later, `ps -ef` shows that the `sleep` process still exists — but its PPID is now `1`. What happened, and why does the system need this mechanism?

Question 3 [Diagnosis]

A colleague shows you this:

```
$ ./deploy.sh
bash: ./deploy.sh: Permission denied
$ echo $?
126
```

The file exists and they own it. Give the exact cause, the command that confirms it, the command that fixes it — and a second way to run the script without fixing anything at all.

Question 4 [Prediction]

Predict the two lines this sequence prints, then explain the difference:

```
MSG=hello
bash -c 'echo 1: $MSG'
export MSG
bash -c 'echo 2: $MSG'
```

Question 5 [Diagnosis]

In an operations script you find `echo $?` printing `137`, right after a Java service died abruptly. Break that number down, say what happened to the process, and explain why this particular code is famous in the container world.

Question 6 [Analysis]

Plenty of impatient administrators reach straight for `kill -9` instead of `kill`. Explain how the two differ mechanically, what a database-style application concretely loses in the second case, and how this relates to the way Docker stops a container (`docker stop`).

Question 7 [Diagnosis]

Look at this:

```
$ cat /etc/shadow
cat: /etc/shadow: Permission denied
$ ls -l /etc/shadow
-rw-r----- 1 root shadow 652 Mar 31 13:31 /etc/shadow
$ sudo cat /etc/shadow    # works
```

Using the `ls -l` line, explain precisely why the first `cat` fails and why the second succeeds. Who could read this file without `sudo`?

Question 8 [Prediction]

Given that `/no-such-path` does not exist, what ends up in `result.txt`, and what appears on screen, after this command?

```
ls /etc/hostname /no-such-path > result.txt 2> errors.txt
```

And what would change if you added `2>&1` after `> result.txt`?

Question 9 [Analysis]

`ss -tlnp` on a server shows these two lines:

```
LISTEN 0  511      127.0.0.1:6379   0.0.0.0:*   users:(("redis-server",pid=812,fd=6))
LISTEN 0  511        0.0.0.0:8080   0.0.0.0:*   users:(("java",pid=944,fd=23))
```

How do these two services differ in reach? Which one can you reach from another machine on the network? And why will this detail matter once you start publishing container ports?

Question 10 [Comprehension]

Your user (UID 1000) runs `python3 -m http.server 80` and gets `PermissionError: [Errno 13] Permission denied` — yet port 8080 works fine. Explain the rule at play, why it exists historically, and what it means for rootless Podman.

Question 11 [Analysis]

`ls /proc` shows hundreds of directories, yet `df -h` reports no disk space used by `/proc`, and `findmnt -t proc` reveals a filesystem of type `proc`. What is `/proc` really? Where do its "files" come from? Give one example of information you would go there to find.

Question 12 [Diagnosis]

On a freshly installed machine, a colleague copied a tool to `~/tools/mytool` and confirmed with `ls` that it is executable. Still:

```
$ mytool
bash: mytool: command not found
$ echo $?
127
```

Explain how the shell searched for `mytool` and why it failed. Then give two permanent fixes — and one immediate workaround.

Question 13 [Analysis]

The "12-factor" methodology insists on configuring applications through **environment variables** rather than hand-edited files. Using what you know about parent-to-child inheritance and the process life cycle, explain why this approach is such a good fit for disposable, restartable processes — which is exactly what your Spring Boot containers will be in lab 08.